Safetrace.

Safetrace is an open-source, full-stack Geographic Information System (GIS) platform for plantation supply-chain traceability. It consists of two independent repositories a **Django-based REST API backend** and a **Next.js frontend dashboard** that together allow organizations to manage farmer data, plantation plots, STDB (Surat Tanda Daftar Budidaya) documents, GAP (Good Agricultural Practice) compliance, and interactive map layers. The project includes a case-study implementation for a farming cooperative in Sekadau Regency, West Kalimantan, Indonesia.

This document consolidates the setup and usage instructions for both the backend and the frontend into a single reference.

Project Overview

Safetrace is designed to give agricultural cooperatives, government agencies, and processing partners a shared, transparent view of where agricultural products come from and how they were produced. The backend exposes a REST API that stores and processes geospatial and compliance data, while the frontend renders that data as an interactive dashboard with maps, tables, and role-based workflows.

Backend (Django)

The backend is a Django-based API that powers all data storage, geospatial processing, and business logic for the traceability system, including deforestation alerts, spatial data handling, and periodic synchronization with external data sources such as Google Earth Engine (GEE).

Backend Dependencies

- Python 3.12
- GDAL library (for shapefile processing)
- Poppler (for `pdf2image`)
- PostgreSQL with the PostGIS extension

- Redis (used as the task queue backend)
- Ubuntu 24.04 (reference operating system)

System Dependencies (Ubuntu)

Ubuntu 24.04 (Python 3.12):

```
bash
sudo apt update
sudo apt install -y python3-dev python3.12-dev
sudo apt install -y gdal-bin libgdal-dev
sudo apt install -y poppler-utils
sudo apt install -y postgresql postgresql-contrib postgis
sudo apt install -y redis-server
```

Backend Setup

1. Clone the repository

```
bash
  git clone <repository-url>
   cd safetrace
```

2. Create a virtual environment

```
bash
  python3.12 -m venv venv
   source venv/bin/activate  # On Ubuntu / Linux
```

3. Install the GDAL Python bindings, matching the installed system library version

```
bash
  # Check the installed GDAL system library version
```

```
gdal-config --version

# Install the Python bindings using the exact same version
pip install gdal==$(gdal-config --version)
```

4. Install the remaining Python dependencies

```
bash
  pip install -r requirements.txt
```

5. Import Indonesian administrative base data Because this project depends on `django-wilayah-indonesia`, the administrative area reference data (provinces, regencies, districts, and villages) must be imported before first use:

```
bash
  ./manage.py import_base_csv
```

6. Run the database migrations

```
bash
  python manage.py migrate
```

7. Start the development server

```
bash
  python manage.py runserver
```

8. Open the application Visit `http://localhost:8000` in your browser.

Production Environment

To run the backend against production settings, export the following environment variable before starting the app:

```
bash
export DJANGO_ENV=prod
```

This tells Django to load its configuration from `settings/prod.py` instead of the default development settings. Make sure a valid production `.env` file is present in the project root before doing this.

Production Setup

1. Clone the project onto the server (recommended location: `/var/www/html`)

```
bash
  cd /var/www/html
   sudo git clone <repository-url> safetrace-backend
   cd safetrace-backend
```

2. **Prepare the production environment file** Ensure the production `.env` file is present in the project root before continuing.

3. Run the initial deployment script

```
bash
  sudo bash scripts/deploy_init.sh
```

4. Validate that the services started correctly

```
bash
  sudo systemctl status safetrace
```

```
sudo systemctl status nginx
```

5. Configure a cron job for the Google Earth Engine (GEE) deforestation data sync Edit the crontab for the user that runs the application (typically `www-data` or your dedicated deploy user):

bash

```
sudo crontab -e
```

Then add a job such as the following, which runs the sync daily at 02:00:

bash

```
0 2 * * * cd /var/www/html/safetrace-backend && DJANGO_ENV=prod
/var/www/html/safetrace-backend/venv/bin/python manage.py
get_data_gee_deforestation >> /var/log/safetrace_gee_deforestation.log 2>&1
```

6. **For subsequent releases and updates**

bash

```
sudo bash scripts/deploy_update.sh
```

Troubleshooting

GDAL Installation Issues

Error: `Python bindings of GDAL X.X.X require at least libgdal X.X.X`

This occurs when the version of the GDAL Python bindings does not match the version of the GDAL system library that is installed. To resolve it:

1. Check the installed GDAL system library version:

bash

```
gdal-config --version
```

2. Install the matching GDAL Python bindings:

```
bash
pip install gdal==$(gdal-config --version)
```

GDAL version reference per environment:

- **Ubuntu 24.04 server:** GDAL 3.8.4 or newer

Error: `Python.h: No such file or directory`

This means the Python development headers are missing. Install them with:

```
bash
# Ubuntu 24.04
sudo apt install python3-dev python3.12-dev
```

Deployment Scripts

Use the provided helper scripts for deployment, both of which automatically install the correct GDAL version based on the system library that is detected:

Initial deployment:

```
bash
sudo bash scripts/deploy_init.sh
```

Update an existing deployment:

```
bash
sudo bash scripts/deploy_update.sh
```

Frontend (Next.js)

The frontend is an open-source GIS dashboard that consumes the backend REST API described above. It can be connected to any compatible backend to manage farmer data, plantation data, STDB documents, GAP compliance records, and interactive map layers.

Key Features

- **Interactive GIS map dashboard** - visualizes plantation plot polygons, deforestation alerts, spatial filters, and custom map layers using Leaflet.
- **Traceability management** - handles farmer records, plantation plots, sales data, and GAP compliance covering production practices, pesticide use, fertilizer use, and hazardous (B3) waste handling.
- **Settings and role-based access control** - supports user management, user profiles, and permission-based roles (Admin, Farmer Group Leader, Government Agency, Factory Partner), as well as map layer configuration.

Frontend Tech Stack

- **Framework:** Next.js 15 (App Router, Turbopack)
- **UI:** React 19 and Tailwind CSS
- **State management:** Redux Toolkit
- **Mapping:** Leaflet, React Leaflet, and Turf.js
- **Tables and charts:** AG Grid, Chart.js, and D3
- **Forms:** Formik and Yup
- **API client:** Axios
- **UI components:** Atomic Design principles combined with Radix UI

Frontend Prerequisites

Before you begin, make sure your system has the following installed:

- Node.js version 18 or newer (version 20 or above is recommended)
- npm (pnpm or yarn are also supported alternatives)

Frontend Setup

1. Clone the repository

```
bash
  git clone https://github.com/adamdavareln/safe-tracibility-system-fe.git
   cd safe-traceability-system-fe
```

2. Install dependencies

```
bash
  npm install
```

3. Configure environment variables Copy the example environment file to create your local `.env` file:

```
bash
  cp .env.local.example .env
Open the `.env` file in your text editor and set the following value:
- `NEXT_PUBLIC_BASE_URL`: the base URL of your backend REST API. This value is required.
```

4. **Run the application**

```
bash
  npm run dev
The application will be available at `http://localhost:3000`.
```

Available Scripts

- `npm run dev` - starts the development server (uses Turbopack for faster builds and refresh times).
- `npm run build` - creates an optimized production build.
- `npm run start` - serves the application from the production build.
- `npm run lint` - runs ESLint to check the codebase for potential errors.

Project Folder Structure

The frontend is built using an Atomic Design architecture to keep the UI component library scalable and maintainable as the project grows.

```
text
src/
├── app/              # Main Next.js App Router directory (pages & layouts)
├── assets/           # Static SVG icons and images specific to the project
├── components/       # Reusable UI components (Atomic Design)
│   ├── atoms/        # Smallest building-block elements (Button, Input, Icon)
│   ├── molecules/    # Combinations of atoms (Form Field, Card Header)
│   ├── organisms/    # Groups of molecules (Modal, Complex Form, Header)
│   ├── providers/    # Context providers (Redux Provider, Toast, Theme)
│   └── ui/           # Components sourced from Radix UI / Shadcn
├── config/           # Theme configuration (brand colors) and base assets
├── constants/        # Global static variables / enums
├── hooks/            # Custom React hooks (data fetching, reusable logic)
├── i18n/             # Localization configuration and translation data
├── libs/             # Utility libraries (permissions, helper logic)
├── services/         # REST API integrations implemented with Axios
├── store/            # Redux state management configuration (slices)
├── styles/           # Global CSS files (globals.css, Tailwind base styles)
├── types/            # Type / interface definitions (for TypeScript portions)
└── utils/            # General-purpose helper functions (formatters, env utilities)

public/               # Publicly served static files (logos, backgrounds, icons)
```

Theme and Logo Customization

1. **Brand colors** - update the theme colors in `src/config/brand.ts`. The corresponding Tailwind classes are generated automatically based on this file.
2. **Logo and assets** - update the asset file paths (institution logo, login background, and others) in `src/config/assets.ts`, then place the corresponding physical files inside the `/public` directory.

License

Both the frontend and backend are released under the MIT License. See the `LICENSE` file in each repository for full details.